Take-Home Assignment — Smart Document Indexing

Candidate: Mehdi Jafari
Role: Senior Prompt Engineer — Smart Document Indexing
Repository: https://github.com/mehdi-jafari/medfar
Code path: `smart-document-indexing/`
Time expectation: 3–4 hours (per assignment brief)

Submission format

Per the MEDFAR take-home assignment (*#0 Instructions - Take Home Assignment*):

> **Format:** Written document of your choice (PDF, Notion, Google Doc, markdown)

This document (`DESIGN.md / DESIGN.pdf`) is the **primary written submission**. The GitHub repository provides runnable prompts, pipeline code, and sample outputs referenced below.

Included with submission:

Deliverable	Location
GitHub repository	https://github.com/mehdi-jafari/medfar
Written assignment (this document)	`DESIGN.pdf`
Prompts	`prompts/01 – 05_*.md`
Pipeline code	`main.py`, `pipeline.py`, `extract.py`, etc.
Output for all sample documents	`outputs/*.json` (generated via `main.py --all`)
Reviewer-friendly samples	`examples/`
Batch metrics	`eval/batch_report.json`

Part 1 — Build the Pipeline (~2 hours)

Context (from assignment)

MEDFAR's Smart Document Indexing processes incoming clinical documents (upload or fax) to determine:

- Document **class and subclass** (MYLE taxonomy)
- **Short description** of content
- **Patient identifying information** present in the document
- **Relevant physician(s)**

Patient matching to EMR records is **out of scope** — extract what is present only.

Output must support downstream **semantic routing rules** (not designed here).

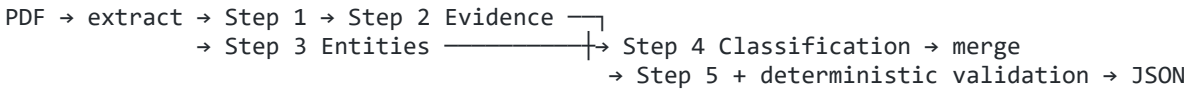
Pipeline architecture — chain vs single prompt

Decision: 5-step prompt chain, orchestrated by `pipeline.py`.

Step	Prompt	Purpose
0	`extract.py`	PDF / fax text extraction (pymupdf → Tesseract → GPT-4o vision)
1	`01_ocr_cleanup.md`	Clean OCR noise; preserve clinical facts
2	`02_evidence_extraction.md`	Extract clues only — no classification
3	`03_entity_extraction.md`	Patient identifiers + physicians
4	`04_taxonomy_classification.md`	MYLE class/subclass + routing text
5	`05_validation.md`	Supplementary LLM review

Why not a single prompt?

A single prompt tends to hallucinate patient data, conflate keywords with document purpose (e.g. "referral" → Consultation Request), and skip explicit ambiguity handling. Separating evidence extraction before classification grounds taxonomy decisions. Isolating entity extraction prevents inventing identifiers during classification.



Handling ambiguous or missing information

Situation	Pipeline behaviour
Missing patient name / de-identified sample	`null` fields + `ambiguities` list
Placeholder physician ("Reading Physician")	Rejected as name; noted in ambiguities
Low classification confidence (< 0.6)	`human_review_required = true`
Invalid MYLE class/subclass pair	Blocking error + review flag
Mixed document types in one PDF	Review flag; single best-fit class (page split not implemented)
Hallucinated field not in source text	`blocking_errors` → `pipeline_status: fail`

Final status fields: `pipeline_status` (`pass` | `pass_with_review` | `fail`), `blocking_errors`, `warnings`, `human_review_required`.

Deterministic validation (`validators.py`) is the primary safety layer. LLM validation (step 5) adds qualitative notes only.

LLM choice

Used: OpenAI **GPT-4o** (via `OPENAI_MODEL` in `.env`).

Why: Sample inputs are scanned faxes and image-only PDFs (prescription form). Strong document understanding and vision fallback are needed when embedded text is absent.

Considered but not benchmarked: Gemini 1.5 Pro, Claude 3.5 Sonnet — same prompt templates could plug in via `llm_client.py`.

Tradeoff: ~5 LLM calls per document (+ vision for image PDFs) — higher cost/latency than a single-call approach.

What I discarded

Tried / considered	Outcome
Single mega-prompt	Hallucinations and misclassification on ambiguous docs
EMR patient matching	Explicitly out of scope
Page-level split indexing	Flagged for review instead; noted as future work
LLM-only validation	Replaced with deterministic checks + supplementary LLM notes
Production OCR vendor	pymupdf + optional Tesseract + vision fallback sufficient for prototype
Fine-tuned classifiers	Out of time; prompt chain evaluable within 3–4 hours

Output for all sample documents

Six sample PDFs provided (assignment references five; six were included in the package). Batch run: **2026-07-09**, model **gpt-4o**.

Document	Status	Predicted class/subclass	Expected	Match
Appointment notice	pass*with*review	Other / Patient Services	Other / Patient Services	Yes
Consultation report	pass*with*review	Clinical Note / Specialist	Clinical Note or Consultation Reports / Specialist	Yes
Imaging (ultrasound)	pass*with*review	Results / Imaging	Results / Imaging	Yes
Prescription	pass*with*review	Requests / Other	Medication and Prescriptions / Prescription Form	No
Referral declined (mixed)	**fail**	Requests / Consultation Requests	Mixed — review required	Partial
Lab result	pass*with*review	Other / Other (0.20 conf)	Results / Laboratory	No

Totals: 3/6 taxonomy correct · 1/6 failed (hallucinated patient name on mixed doc) · Full JSON in `outputs/`.

Prompts: `prompts/`. Example outputs (no API key): `examples/`.

Part 2 — Evaluation Rubric (~45 min)

Rubric 1 — Taxonomy accuracy

Definition: The assigned MYLE class/subclass correctly reflects the document's primary clinical or administrative purpose.

Strong	Correct class/subclass with confidence ≥ 0.8; reasoning aligns with document purpose (not keywords alone).
Weak	

	Wrong bucket (e.g. appointment confirmation classified as Consultation Request) or invalid taxonomy pair.

How to evaluate: Gold labels per sample (`eval/labels/`); batch report classification accuracy. **Tradeoff:** Gold labels are manual and small-n; cheap to run but not statistically robust. Alternative: clinician adjudication panel — accurate but slow and expensive.

Rubric 2 — Extraction fidelity (no hallucination)

Definition: Patient identifiers and physician names appear in the source document and are not invented.

Strong	Fields match source text; missing data returned as `null` with ambiguities noted.
Weak	Name or identifier present in JSON but absent from document text.

How to evaluate: Deterministic fuzzy match of extracted fields against cleaned source text (`validators.py`). **Tradeoff:** Fuzzy matching tolerates OCR variation but may miss subtle formatting differences; strict exact match would increase false failures.

Rubric 3 — Safety / human review appropriateness

Definition: The pipeline escalates ambiguous, mixed, or low-confidence documents to human review; clear documents are not unnecessarily blocked.

Strong	Mixed referral doc flagged; imaging report with clear class routes without false `fail` .
Weak	Silent auto-approval of wrong class, or excessive review flags on straightforward docs.

How to evaluate: Compare `human_review_required` and `pipeline_status` against gold `human_review_required` ; measure review-flag precision/recall on labeled set. **Tradeoff:** Optimizing for recall (catch all bad docs) increases human workload; optimizing for precision reduces triage burden but risks missed errors.

Rubric 4 — Routing support usefulness

Definition: `routing_support_text` and physician roles provide a concise, factual summary usable by semantic routing rules.

Strong	Short, specific summary (e.g. "Specialist appointment notice, consultation Apr 29 2026"); physician role supports `document_physician` routing.
Weak	Generic text, hallucinated content, or placeholder physician name used as routing target.

How to evaluate: Manual review against assignment routing rule structure; optional embedding similarity to clinician-written queries. **Tradeoff:** Manual review is qualitative; automated semantic similarity needs a query benchmark not available in this exercise.

Implementation: `eval/scorer.py` , `eval/run_eval.py` , Streamlit dashboard (`eval/app.py`).

Part 3 — Failure Modes & Validation (~60 min)

Failure mode 1 — Keyword-driven misclassification

What it looks like: Document classified by surface words rather than purpose. Example: appointment notice with "referral" and "consultation" → `Requests / Consultation Requests` instead of `Other / Patient Services`.

When it occurs: Administrative notices, scheduling letters, declined referrals with clinical vocabulary; any doc where taxonomy labels overlap with document language.

How to test systematically:

- Labeled set with purpose-based gold classes (not keyword labels)
- Batch eval per document type; track confusion matrix between `Requests` vs `Other / Patient Services`
- Adversarial samples: confirmation letters that mention "request" or "referral"

Redesign (most clinically significant for routing):

- Add purpose-based rules in classification prompt (confirmations ≠ requests)
- Require step 2 evidence to include `document_type_clues` before step 4 classifies
- Lower auto-route confidence threshold; require `human_review_required` when evidence conflicts with class
- Deterministic check: if `appointment_or_administrative_clues` present without `request_clues`, block `Requests/*` unless confidence very high

Failure mode 2 — Hallucinated or placeholder entities

What it looks like: Patient name in JSON not on document; physician recorded as "Reading Physician" instead of `null` + ambiguity.

When it occurs: De-identified samples, image-only PDFs, imaging reports with role labels only; LLM fills gaps when fields expected but absent.

How to test systematically:

- Deterministic validation: every non-null patient field must fuzzy-match source text
- Placeholder name denylist (`validators.py`)
- Inject de-identified documents; assert `name: null` and non-empty `ambiguities`
- Track `blocking_errors` rate on gold set

Redesign:

- Post-process entities through `sanitize_physicians()` — reject role labels
- `pipeline_status: fail` when hallucination detected (blocks silent auto-indexing)
- Entity prompt explicitly forbids role labels as names
- Human review step mandatory when any identifier field fails source-text check

---

MYLE taxonomy reference

Mapped in `schema.py` per assignment taxonomy. Notable: `Summary` includes **Record Summary Request** (single subclass).

Limitations (prototype scope)

- Six samples only; not production-ready OCR
- Single class per PDF; mixed documents flagged not split
- English/French mixed content without dedicated bilingual handling
- 5 LLM calls per document — cost and latency; **single model (`gpt-4o`) for all steps** in this prototype
- Assignment time box (3–4 hours) — depth over exhaustive coverage

Future work (see ROADMAP.md)

- **Per-step model routing:** Benchmark API models (gpt-4o, gpt-4o-mini) and open-source/local models (e.g. Llama/Mistral via Ollama) per pipeline step using the eval harness. Choose the cheapest model per step that meets KPI floors (e.g. no hallucinated entities on step 3, ≥85% taxonomy accuracy on step 4). Keep the strongest model on entity extraction and classification until eval proves a cheaper alternative is safe.
- Page-level segmentation for mixed PDFs
- Scale labeled benchmark dataset (6 → 200+ documents)

Reproduce results

```
git clone https://github.com/mehdi-jafari/medfar.git
cd medfar/smart-document-indexing
copy .env.example .env   # OPENAI_API_KEY required
.\.venv\Scripts\pip install -r requirements.txt
.\.venv\Scripts\python main.py --all
.\.venv\Scripts\python eval/run_eval.py
```

Repository: https://github.com/mehdi-jafari/medfar · Setup details: `README.md` · Future work: `ROADMAP.md`.